

UNIVERSIDADE FEDERAL DE UBERLÂNDIA
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

ANA LUIZA LISBOA MOURA
ANTÔNIO COUTINHO ABREU VELOSO
GUILHERME DE SOUZA MOTA

TRABALHO FINAL
ALGORITMOS E ESTRUTURAS DE DADOS I

UBERLÂNDIA

2026

ANA LUIZA LISBOA MOURA
ANTONIO COUTINHO ABREU VELOSO
GUILHERME DE SOUZA MOTA

TRABALHO FINAL
ALGORITMOS E ESTRUTURAS DE DADOS I

Trabalho tem como objetivo relatar a criação de um código elaborado para resolução do problema proposto na disciplina de Algoritmos e Estruturas de Dados I do curso de Bacharelado em Ciência da Computação.

Docente: Profa. Maria Adriana Vidigal de Lima

UBERLÂNDIA

2026

SUMÁRIO

1 INTRODUÇÃO.....	4
2 DOCUMENTAÇÃO DO CÓDIGO	6
2.1 FUNÇÕES DAS OPERAÇÕES BÁSICAS DE REGIÕES.....	6
2.2 FUNÇÕES DAS OPERAÇÕES BÁSICAS DE VINÍCOLAS.....	14
2.3 FUNÇÕES ADICIONAIS	21
2.4 FUNÇÃO MAIN E MENUS.....	26
3 EXEMPLOS DE USO	37
4 CONCLUSÃO	42

INTRODUÇÃO

A resolução do trabalho proposto se iniciou com a escolha do tema. Dentre os disponíveis, o grupo, em consenso, escolheu o tema 14 – Regiões e Vinícolas.

Em primeira análise, o problema exigia que fossem criadas duas camadas de percurso: uma camada principal, que permita transitar entre os seus elementos, e uma camada secundária, que admita acessar os elementos secundários que estão associados a cada elemento da camada principal. Dessa forma, compreendeu-se que a solução seria confeccionar uma lista de sublistas.

Diante disso, a solução do problema foi desenhada a partir desses dois níveis de encadeamento. Considerando o tema escolhido, elaborou-se uma lista principal responsável por guardar as Regiões e, ligadas à cada Região, listas secundárias responsáveis por armazenar as Vinícolas.

Para tal, utilizamos três estruturas de dados do tipo *struct* para arquitetar as listas e seus elementos. A primeira estrutura, apelidada de “Vinícolas”, armazena os dados de cada vinícola – nome, produtos, município e ano de fundação – além de dois ponteiros que guardam os endereços de outras estruturas do tipo Vinícolas: um que aponta para a vinícola anterior e outro que aponta para a próxima vinícola, caracterizando, assim, o encadeamento duplo que estrutura o padrão das sublistas.

```
#define MAX 1000

typedef struct vinicolas {
    char nomeVin[MAX];
    char produtos[MAX];
    char municipio[MAX];
    int anoFund;

    struct vinicolas *ant; // encadeamento das vinicolas
    struct vinicolas *prox;
} Vinicolas;
```

Já a segunda estrutura, denotada por “Regiões”, foi criada para guardar as informações de cada região – nome e estado - e os dois ponteiros que guardam os

endereços de outras estruturas do tipo Regiões: um que aponta para a região anterior e outro que aponta para a próxima região, estruturando o encadeamento duplo da lista principal; e, ainda, essa estrutura também foi usada como descritor para a lista secundária correspondente àquela região, o que permitiu armazenar a quantidade de elementos, o início e o final da sublista de vinícolas associada.

```
typedef struct regioes {  
  
    char nomeReg[MAX];  
    char estado[MAX];  
    struct regioes *anterior; // encadeamento das regioes  
    struct regioes *proximo;  
  
    Vinicolas *init; // descritor para a lista secundaria, dentro da regioao  
    Vinicolas *final;  
    int quantVin; // quantidade de vinicolas na regioao  
}Regioes;
```

Enfim, a terceira estrutura, denominada por “ListaRegioes”, faz o papel de descritor da lista principal, guardando a quantidade de regiões cadastradas, o início e o final da lista.

```
typedef struct listaregioes { // descritor de regioes  
    Regioes *inicio;  
    Regioes *fim;  
    int quant;  
}ListaRegioes;
```

Portanto, a resolução do problema se concretizou a partir dessas três estruturas, que permitem que manipulemos todo o sistema. Desse modo, é possível percorrer as regiões e, de cada região, transitar entre as vinícolas nela cadastradas, configurando assim uma lista principal duplamente encadeada de listas secundárias duplamente encadeadas.

DOCUMENTAÇÃO DO CÓDIGO

FUNÇÕES DAS OPERAÇÕES BÁSICAS DE REGIÕES

Para criar uma lista vazia de regiões, implementa-se uma função de nome “criarListaVazia”, que devolve um ponteiro para uma struct do tipo ListaRegioes. Nela, aloca-se o espaço de memória para essa lista, testa-se se a alocação foi bem sucedida, adequa-se os campos quant para 0 e o início e o fim apontando para NULL e retorna-se o ponteiro l, um descritor de uma lista principal vazia.

```
ListaRegioes *criarListaVazia ()
{
    ListaRegioes *l = (ListaRegioes*) malloc (sizeof(ListaRegioes));
    if (l == NULL) {
        return NULL; // falha de alocação
    }

    l->quant = 0;
    l->inicio = NULL;
    l->fim = NULL;
    return l;
}
```

Agora, para facilitar a inserção na lista, mantendo a função de inserir apenas com o objetivo de inserir e não criar e inserir uma região, implementou-se a função de nome “criarRegiao” para chamá-la no momento da inserção. A função recebe duas strings – nome e estado da região - e retorna um ponteiro para uma struct do tipo Regioes. Primeiro, aloca-se espaço de memória para uma região r, verifica-se se a alocação foi bem sucedida e adequa-se os campos de r, passando o nome e o estado, apontando todos os ponteiros da estrutura para NULL e zerando a quantidade de vinícolas da região e, no final, retorna-se a região criada. Vale destacar que ao criar uma região também criamos uma lista de vinícolas, já que uma estrutura de região é um descritor de uma lista de vinícolas.

```

Regioes *criarRegiao (char *c, char *n) // recebe o nome e estado da regioao
{
    Regioes *r = (Regioes*) malloc (sizeof(Regioes)); // aloca espaço pra regioao
    if (r == NULL)
    {
        return NULL; // falha de alocação
    }

    strcpy(r->nomeReg, n);
    strcpy(r->estado, c);
    r->anterior = NULL;
    r->proximo = NULL;
    r->init = NULL;
    r->final = NULL;
    r->quantVin = 0;
    return r; // retorna a regioao
}

```

Para inserir regiões na lista principal, implementou-se a função de nome “inserirInicio” do tipo void recebe o descritor de uma lista de regiões (l), e duas strings - o nome e o estado da região. Primeiramente, a função chama a “criarRegiao”, passando as duas strings como parâmetros para criar uma nova região. Após, verifica-se se a criação foi bem sucedida. Para a inserção, se a lista for vazia, o início e o fim dela será a nova região r, caso contrário, inserimos no início, o próximo de r será o antigo início (l->inicio), o anterior do início será o r e então, o r vira o início da lista. Por fim, atualiza-se a quantidade de regiões.

```

void inserirInicio (ListaRegioes* l, char *c, char *n){
    Regioes *r = criarRegiao(c, n); // cria a regioao com os parametros

    if (r==NULL)
    {
        return;
    }

    if (l->quant == 0){ // se a lista for vazia
        l->inicio = r;
        l->fim = r;
    } else {

        r->proximo = l->inicio; // se nao for vazia
        l->inicio->anterior = r;
        l->inicio = r;
    }

    l->quant++;
}

```

Para buscar dada região pelo seu nome na lista principal, implementou-se a função de nome “buscaPorNomeReg” que recebe a lista de regiões e uma string e que devolve um ponteiro para struct do tipo Regioes. Se a lista for vazia, a função retorna NULL e um aviso na tela, caso contrário, cria-se um ponteiro auxiliar que aponta para o início da lista e percorre até o final. Em meio ao percurso, verifica-se se o nome de alguma região é igual a string fornecida, em caso afirmativo, o elemento foi encontrado e é impresso na tela e retornado pela função; caso saia do laço do percurso, a região não foi encontrada na lista, assim, imprime-se essa informação na tela e retorna-se NULL;


```

Regioes *buscaPorNomeReg (ListaRegioes *l, char *n)
{
    if (l->quant==0) // se a lista for vazia
    {
        printf ("Lista Vazia!\n");
        return NULL;
    }
    Regioes *aux = l->inicio; // aux para percorrer
    while (aux != NULL)
    {
        if( strcmp (aux->nomeReg,n)==0) // nome igual ao que estamos procurando
        {
            printf ("Região %s encontrada!\n", n); // se achar
            return aux;
        }
        aux = aux->proximo;
    }
    printf ("Região %s nao encontrada!\n", n); // percorreu tudo e nao achou
    return NULL;
}

```

Para alterar dados de uma região já existente, a função do tipo void chamada “alteraDadosReg” recebe a lista de regiões e uma string. Nela, verifica-se se a lista está vazia, caso não estiver, cria-se um ponteiro auxiliar e chama-se a função de busca por nome - passando a string e a lista como parâmetros -, armazena-se a região que queremos alterar no aux. Se aux for diferente de NULL significa que a região existe e, portanto, altera-se os dados, caso contrário (se for NULL), a função de busca irá avisar que a região não foi encontrada e a função de alterar dados encerrará.

```

void alteraDadosReg (ListaRegioes *l, char *nomeAntigo)
{
    if (l->quant==0) // lista vazia
    {
        printf ("Lista Vazia!\n");
        return;
    }
    Regioes *aux = buscaPorNomeReg(l,nomeAntigo); // encontrar a regioao alvo
    if (aux!=NULL) // se a busca tiver sucesso
    {
        char nomeNovo[MAX];
        char estadoNovo[MAX];

        printf("Digite o novo nome: ");
        scanf("%s", nomeNovo); // lemos o novo nome

        printf("Digite o novo estado: ");
        scanf("%s", estadoNovo); // lemos o novo estado

        strcpy(aux->nomeReg, nomeNovo); // atualizamos os campos
        strcpy(aux->estado, estadoNovo);
    } else { // se não for encontrado
        return;
    }
}

```

Para remover uma região, a função de nome “removerReg” recebe a lista de regiões e o nome de uma região. Primeiro, ela verifica se a lista está vazia e, se estiver, avisa na tela e se encerra, caso contrário, a função cria um ponteiro auxiliar para apontar para a região que desejamos deletar, utilizando a função de busca por nome para encontrá-la. Caso a busca falhe, a função se encerra. Se não falhar, devemos analisar alguns casos:

- 1- Se a lista tiver somente um elemento: o início e o fim devem apontar para NULL, removendo o elemento da lista;
- 2- Se o elemento estiver no início: o início da lista deve apontar para o segundo da lista, “pulando” a região que queremos apagar, e o anterior do novo início devem apontar para NULL;

- 3- Se o elemento estiver no final: o final deve virar o penúltimo, e o próximo dele deve apontar para NULL, ideia semelhante ao elemento no início;
- 4- Se estiver no meio da lista: “pulamos” o aux na lista, adequando os ponteiros anterior e próximo do elemento anterior e próximo do aux.

Após adequar os ponteiros de acordo com o caso que a região se encontra, a função libera, primeiramente, as vínculas vinculadas à região, criando um ponteiro auxVin que aponta para o início da lista e percorre toda a lista liberando todas as vínculas para não gerar lixo de memória. Ao fim, libera-se o aux e atualiza-se a quantidade de regiões da lista.

```
111 void removerReg(ListaRegioes *l, char* nome)
112 {
113     if (l->quant==0) // lista vazia
114     {
115         printf ("Lista Vazia!\n");
116         return;
117     }
118
119     Regioes *aux = buscaPorNomeReg(l,nome); // buscar a região
120
121     if (aux == NULL) // se a busca não encontrar
122     {
123         return;
124     }
125
126     if (l->quant == 1)
127     {
128         l->inicio = NULL;
129         l->fim = NULL;
130     }
131
```

```
130     }
131
132     else if (aux == l->inicio) // se o elemento for o início da lista
133     {
134
135         l->inicio = l->inicio->proximo; // inicio vira o segundo
136         l->inicio->anterior = NULL;
137
138     }
139     else if (aux == l->fim) // se for o fim da lista
140     {
141         l->fim = l->fim->anterior; // fim vira o penultimo
142         l->fim->proximo = NULL;
143     }
```

```

143     }
144     else // se tiver no meio da lista
145     {
146         aux->anterior->proximo = aux->proximo;
147         aux->proximo->anterior = aux->anterior; // ("pulamos" o aux na lista, arrumando os ponteiros)
148     }
149
150     Vinicolas *auxVin = aux->init; // libera a região das vinicolas da região
151     while (auxVin != NULL)
152     {
153         Vinicolas *prox = auxVin->prox; // guardamos o prox
154
155         free(auxVin);
156
157         auxVin = prox;
158     }
159
160     printf("Região %s removida.\n", nome);
161
162     free (aux); // liberamos a memória do aux
163     l->quant--;
164 }
165

```

Para listar as regiões existentes, a função de tipo void de nome “listarRegioes” recebe uma lista. Se a lista for vazia, é impresso um aviso na tela e a função se encerra. Caso contrário, percorre-se toda a lista com um ponteiro auxiliar e imprime-se, através de um laço, o nome e o estado de todas as regiões da lista.

```

void listarRegioes (ListaRegioes* l){

    if (l->quant==0) // se a lista for vazia
    {
        printf ("Lista Vazia!\n");
        return;
    }
    Regioes *aux = l->inicio; // aux pra percorrer ate o fim
    while (aux!=NULL)
    {
        printf ("Nome: %s\n", aux->nomeReg);
        printf ("Estado: %s\n\n", aux->estado);

        aux = aux->proximo;
    }
    printf ("Lista finalizada!\n\n");
}

```

Para saber a quantidade de regiões presentes na lista, a função de tipo int denominada por “contaQuantReg”. Como nossa estrutura de dados ListaRegioes tem o campo quant que guarda o número de regiões, a função a penas recebe a lista e retorna esse campo.

```
int contaQuantReg (ListaRegioes *l)
{
    return l->quant;
}
```

FUNÇÕES DAS OPERAÇÕES BÁSICAS DE VINÍCOLAS

Para facilitar a inserção de vinícolas, implementa-se a função de criar vinícola de nome “criarVin”, que recebe três strings e um inteiro. Nela, aloca-se a memória necessária para a nova vinícola, verifica-se se a alocação deu certo. Em seguida, preenche-se os campos da estrutura com os parâmetros da função - nome, produtos, município e ano de fundação - e aponta-se os ponteiros de encadeamento para NULL inicialmente, retornando a vinícola nova no final.

```
Vinícolas *criarVin(char *n, char *p, char *m, int ano)
{
    Vinícolas *novo = (Vinícolas*) malloc (sizeof(Vinícolas)); // alocação de memória para a vinícola

    if (novo == NULL)
    {
        return NULL;
    }

    strcpy(novo->nomeVin, n);
    strcpy(novo->produtos, p);
    strcpy(novo->município, m);
    novo->anoFund = ano;
    novo->ant = NULL;
    novo->prox = NULL;

    return novo;
}
```

Para inserção de uma vinícola no início da lista, implementa-se a função de tipo void nomeada por “inserirVinInicio”, que recebe uma região (consequentemente, uma lista de vinícolas) e os parâmetros necessários para criar uma vinícola. Cria-se a vinícola através da função de criar vinícola já implementada, passando todos os parâmetros necessários. Após, verifica-se se a criação foi bem sucedida, se sim, devemos analisar dois casos:

1. Lista está vazia: o fim e o início (final e init) devem apontar para a nova vinícola;
2. Lista não vazia: por meio dos ponteiros, insere-se o novo antes do antigo início e atualiza-se o início (init), apontando-o para novo.

Após isso, acrescentamos a quantidade de vinícolas na lista.

```

void inserirVinInicio(Regioes* r, char *n, char *p, char *m, int ano){

    Vinicolas *novo = criarVin(n, p, m, ano); // criamos a vinicola

    if (novo == NULL)
    {
        return;
    }

    if (r->quantVin == 0){ // lista vazia
        r->init = novo;
        r->final = novo;
    } else {

        novo->prox = r->init; // inserimos no inicio
        r->init->ant = novo;
        r->init = novo;
    }

    r->quantVin++;
}

```

Para buscar uma vinícola pelo nome, implementa-se a função de nome “buscaPorNomeVin”, que recebe uma região (que também é uma lista de vinícolas) e uma string representando o nome da vinícola que se busca e que retorna um ponteiro para uma estrutura do tipo Vinícolas. Se a lista for vazia, a função exibe um aviso e se encerra, caso contrário, ela cria um ponteiro auxiliar para percorrer toda a lista, comparando o nome de todas as vinícolas até encontrar a vinícola em questão. Caso encontre no meio do percurso, a função exibe uma mensagem e se encerra, retornando a vinícola; caso contrário, o laço perdura até que se chegue ao final da lista, se chegar no final e sair do laço significa que a vinícola não foi encontrada, assim, a função retorna NULL e uma mensagem de aviso.

```

Vinicolas *buscaPorNomeVin (Regioes *r, char *n)
{
    if (r->quantVin==0) // lista vazia
    {
        printf ("Nenhuma vinicola na regioao!\n");
        return NULL;
    }

    Vinicolas *aux = r->init; // aux para percorrer

    while (aux != NULL)
    {
        if( strcmp (aux->nomeVin,n)==0) // comparamos o nome com o alvo
        {
            printf ("Vinicola %s encontrada!\n", n);
            return aux;
        }
        aux = aux->prox;
    }
    printf ("Vinicola %s nao encontrada!\n", n); // se saiu do laço a vinicola n foi encontrada
    return NULL;
}

```

Para alterar dados de alguma vinícola já existente, implementa-se uma função do tipo void de nome “alteraDadosVin”, que recebe uma região (lista de vinícolas) e o nome de uma vinícola (string). Se a região não tiver vinícolas, a função exibe uma mensagem e se encerra, caso contrário, cria-se um ponteiro auxiliar para guardar a vinícola que será buscada através de “buscaNomeVin”, passando a mesma região e o nome recebidos. Se encontrar, lê os novos dados e realiza a alteração, se não encontrar, a função se encerra e a mensagem de erro será dada pela própria função de busca.


```

void alteraDadosVin (Regioes *r, char *nomeAntigo)
{
    if (r->quantVin==0) // lista vazia
    {
        printf ("Nenhuma vinicola na regioao!\n");
        return;
    }
    Vinicolas *aux = buscaPorNomeVin(r,nomeAntigo); // buscamos a vinicola pelo nome

    if (aux!=NULL)
    {
        char nomeNovo[MAX];
        char produtosNovos[MAX];
        char muniNovo[MAX];
        int anoNovo;

        printf("Digite o novo nome: ");
        scanf(" %s", nomeNovo);

        printf("Digite os novos produtos: ");
        scanf(" %s", produtosNovos);

        printf("Digite o novo municipio: ");
        scanf(" %s", muniNovo);

        printf("Digite o novo ano: ");
        scanf(" %d", &anoNovo);

        strcpy(aux->nomeVin, nomeNovo);
        strcpy(aux->produtos, produtosNovos);
        strcpy(aux->municipio, muniNovo);
        aux->anoFund = anoNovo;
    } else {
        return;
    }
}

```

Para remover uma vinícola, implementa-se uma função de tipo void nomeada de “removerVin”, que recebe uma região e um nome de vinícola. Se não houver nenhuma vinícola na região, a função exibe um aviso e encerra, caso contrário criamos um ponteiro auxiliar para armazenar a vinícola utilizando buscaPorNomeVin. Se a busca falhar, a função se encerra, caso contrário, devemos analisar alguns cenários:

1. Se a lista tiver somente um elemento: o elemento em questão é o que estamos procurando, já que a busca não falhou, assim, o início e o final da lista devem apontar para NULL, excluindo a vinícola da lista;
2. O elemento está no início da lista: devemos “pular” ele, fazendo com que o início seja o segundo elemento;
3. O elemento está no final da lista: semelhante ao início, devemos fazer com que o final agora seja o penúltimo elemento;
4. Se estiver no meio da lista: também devemos “pular” ele, arrumando os ponteiros ant e prox dos elementos anterior e próximo do aux.

Uma vez que o caso seja realizado, libera-se o aux e se atualiza a quantidade de vinícolas da região.

```
293
294 void removerVin(Regioes *r, char* nome)
295 {
296     if (r->quantVin==0)
297     {
298         printf ("Nenhuma vinicola na região!\n"); // lista vazia
299         return;
300     }
301
302     Vinicolas *aux = buscaPorNomeVin(r,nome);
303
304     if (aux == NULL) // nao achou a vinicola na regioao
305     {
306         return;
307     }
308
309     if (r->quantVin == 1) // uma vinicola
310     {
311         r->init = NULL;
312         r->final = NULL;
313     }
314
```

```

314
315     else if (aux == r->init) // vinicola no inicio
316     {
317
318         r->init = r->init->prox;
319         r->init->ant = NULL;
320
321     }
322     else if (aux == r->final) // vinicola no final
323     {
324         r->final = r->final->ant;
325         r->final->prox = NULL;
326     }
327     else // vinicola no meio
328     {
329         aux->ant->prox = aux->prox;
330         aux->prox->ant = aux->ant;
331     }
332
333     printf ("Vinicola %s removida com sucesso.\n", nome);
334     free (aux); // liberamos a vinicola
335     r->quantVin--;
336 }

```

Para listar todas as vinícolas de uma região, implementa-se uma função de tipo void nomeada por “listarVin”, que recebe uma região. Se não houver vinícolas, ela exibe uma mensagem e se encerra, caso contrário, percorre-se toda a lista com um ponteiro auxiliar e se exibe todas as informações de todas as vinícolas uma a uma, caminhando com o auxiliar dentro de um laço até o final. Ao fim, a função exibe uma mensagem de finalização.

```

void listarVin (Regioes* r){
    if (r->quantVin==0)
    {
        printf ("Nao ha vinicolas!\n");
        return;
    }
    Vinicolas *aux = r->init;
    while (aux!=NULL)
    {
        printf ("Nome da vinicola: %s\n", aux->nomeVin);
        printf ("Produtos: %s\n", aux->produtos);
        printf ("Municipio: %s\n", aux->municipio);
        printf ("Ano de fundacao: %d\n\n", aux->anoFund);
        aux = aux->prox;
    }
    printf ("Lista finalizada!\n\n");
}

```

Para saber a quantidade de vinícolas presentes na lista de vinícolas (região), a função de tipo int denominada por “contaQuantVin”. Como nossa estrutura de dados Regioes tem o campo quantVin que guarda o número de vinícolas associadas à região, a função apenas recebe a lista e retorna esse campo.

```
int contaQuantVin (Regioes *l)
{
    return l->quantVin;
}
```

FUNÇÕES ADICIONAIS

A primeira função adicional implementada foi criada pelos membros do grupo. A função de busca geral tem como objetivo encontrar uma vinícola em alguma região apenas pelo seu nome, sendo necessário percorrer todas as regiões e buscar a vinícola em cada uma. Ela recebe uma lista de regiões e uma string (nome da vinícola), se a lista for vazia retorna nada, caso contrário, criamos um auxiliar do tipo regiões para percorrer toda a lista de regiões. Dentro de cada região, chamamos a função buscaPorNomeVin, passando a região aux e o nome da vinícola. Caso a vinícola seja encontrada na região, a função exibe uma mensagem e se encerra, caso contrário, avança-se o aux para o seu próximo. Caso saia do laço significa que a vinícola não foi encontrada em nenhuma região, o que é impresso pela função antes de se encerrar por completo.

```
void buscaGeral (ListaRegioes *r, char *nome){  
    if (r == NULL) {  
        return;  
    }  
  
    Regioes *aux = r->inicio;  
  
    while (aux != NULL){ // percorrer toda a lista de regioes  
        Vinicolas *v = buscaPorNomeVin(aux, nome); // buscar dentro de cada regioao  
  
        if (v != NULL)  
        {  
            printf ("Vinicola %s encontrada na regioao %s\n", nome, aux->nomeReg);  
            return;  
        }  
        aux = aux->proximo;  
    }  
  
    printf ("Vinicola nao encontrada em nenhuma regioao.\n");  
    return;  
}
```

A segunda função adicional também foi criada pelos membros do grupo, seu objetivo é contar quantas vinícolas existem por região. Por mais que exista o campo QuantVin na nossa estrutura Região, fizemos uma função que conte a quantidade de vinícolas dentro de uma região, percorrendo a lista inteira e retornando a quantidade de vinícolas em cada região. Assim, a função recebe uma lista de regiões, se estiver vazia encerramos a função, caso contrário, criamos um ponteiro auxiliar para as regiões (auxReg) que usaremos para percorrer toda a lista e um inteiro contadorVin. Percorrendo a lista, acessamos cada região e criamos um auxiliar para vinícolas (auxVin) para percorrer toda a lista de vinícolas de uma região, cada vez que acessarmos uma vinícola, adicionamos 1 ao contadorVin e caminhamos para a próxima vinícola, ao sair desse laço, exibimos quantas vinícolas tem na região, andamos com o auxReg para a próxima região e zeramos o contador, entrando em uma nova região e refazendo todo o processo até o fim da lista de regiões.

```
void contaVinPorReg (ListaRegioes *r){  
  
    if (r == NULL) {  
        return;  
    }  
  
    Regioes *auxReg = r->inicio;  
    int contadorVin = 0;  
  
    while (auxReg != NULL){  
  
        Vinicolas *auxVin = auxReg->init;  
  
        while (auxVin != NULL) {  
  
            contadorVin++;  
            auxVin = auxVin->prox;  
  
        }  
        printf ("Regiao %s : %d Vinicolas\n", auxReg->nomeReg, contadorVin);  
        auxReg = auxReg->proximo;  
        contadorVin = 0;  
    }  
  
    return;  
}
```

A terceira função adicional foi disponibilizada como exemplo pela docente e tem como objetivo encontrar a região que possui a menor quantidade de vinícolas associadas. A função recebe uma lista de regiões e retorna um ponteiro para uma estrutura do tipo Regioes que, nesse caso, representará a região com menor número de vinícolas. Quando essa lista de regiões for vazia, retornamos NULL, pois

não há nenhuma região para ser analisada. Por outro lado, quando há uma ou mais regiões na lista, criamos um ponteiro auxiliar que aponta para a primeira região, outro ponteiro que guardará o endereço da região com menos vinícolas e uma variável (inicializado com -1 garantindo que a primeira região receba o posto de menor) que guardará a menor quantidade, até então, de vinícolas. A ideia é percorrer todas as regiões contando o número de vinícolas através de um laço e, a cada região que se passa, compara-se o número guardado no contador atual com o menor número já registrado; quando, no percurso da lista de regiões, a função encontrar um número de vinícolas menor do que o número armazenado como o menor, o ponteiro menor apontará para essa região com menos vinícolas e a variável menorCont atualizará para quantia dessa região. Ao final, ponteiro menor apontará para a região que, dentre todas, teve o menor contador. A função retorna uma mensagem indicando o nome e a quantidade de vinícolas da região reservado em no ponteiro menor e, também, retorna a própria região.

```
Regioes *menorQuantVin(ListaRegioes *l)
{
    if (l == NULL || l->quant == 0) {
        return NULL;
    }

    Regioes *aux = l->inicio;
    Regioes *menor = NULL;
    int menorCont = -1;

    while (aux != NULL) {
        Vinicolas *auxVin = aux->init;
        int contador = 0;

        while (auxVin != NULL) {
            contador++;
            auxVin = auxVin->prox;
        }

        if (menor == NULL || contador < menorCont) {
            menor = aux;
            menorCont = contador;
        }

        aux = aux->proximo;
    }

    printf("Regiao com menos vinicolas: %s (%d vinicola(s))\n", menor->nomeReg, menorCont);
    return menor;
}
```

A quarta função adicional também foi disponibilizada como exemplo pela docente e tem como objetivo filtrar todas as vinícolas de uma região que pertencem ao mesmo município. A função de tipo void recebe uma região (lista de vinícolas) e uma string representando o município que se deseja filtrar. Quando a lista de vinícolas for vazia, a função se encerra. Quando há elementos na lista, a função utiliza um ponteiro auxiliar que aponta para a primeira vinícola e que será utilizado para percorrer a lista. Repare que foi declarada uma variável de teste inicializada com 0. No decorrer do percurso, todas as vinícolas que tiverem o campo município igual a string fornecida serão impressas na tela. Caso nenhuma vinícola corresponda ao filtro, a função não entra na condição dentro laço e, conseqüentemente, a variável de teste permanece com o valor 0, o que será crucial para adicionar outra condição ao final da função que imprime uma mensagem de aviso se não houve nenhuma vinícola filtrada.

```
void filtrarVinPorMunicipio(Regioes *r, char *nome)
{
    if (r == NULL) {
        return;
    }

    Vinicolas *aux = r->init;
    int testeVazio = 0;

    while (aux != NULL) {
        if (strcmp(aux->municipio, nome) == 0) {
            printf("Nome da Vinicola: %s\n", aux->nomeVin);
            testeVazio = 1;
        }
        aux = aux->prox;
    }

    if (testeVazio == 0) {
        printf("Nenhuma vinicola encontrada no municipio %s.\n", nome);
    }
    return;
}
```

A quinta função adicional também foi disponibilizada como exemplo pela docente e tem como objetivo contabilizar as vinícolas de uma região que pertencem ao mesmo município. A função de tipo void recebe uma região (lista de vinícolas) e

uma string representando o município que se deseja usar para contabilização. Quando a lista de vinícolas for vazia, a função se encerra. Quando há elementos na lista, a função utiliza um ponteiro auxiliar que aponta para a primeira vinícola e uma variável contador inicializada em 0. A ideia é percorrer a lista e incrementar o contador sempre que o município da vinícola em atual análise for igual ao nome dado a função. Caso nenhuma vinícola corresponda, o contador continua com valor 0 e a função imprime que não há nenhuma vinícola nesse município, caso contrário, a função imprime uma mensagem junto com a variável contador, indicando o número de vinícolas que pertencem a tal município.

```
void contarVinPorMunicipio(Regioes *r, char *nome)
{
    if (r == NULL)
    {
        return;
    }
    Vinicolas *aux = r->init;
    int contador=0;

    while (aux != NULL) {
        if (strcmp(aux->municipio, nome) == 0) {
            contador++;
        }
        aux = aux->prox;
    }

    if (contador == 0) {
        printf("Nenhuma vinicola no municipio %s.\n", nome);
    }
    else {
        printf ("Numero de Vinicolas no municipio: %d\n",contador);
    }
    return;
}
```

FUNÇÃO MAIN E MENUS

O primeiro ato da função main é declarar um ponteiro para estrutura do tipo ListaRegioes nomeado de “lista” e chamar a função criarListaVazia. Logo após, para preencher a lista mais facilmente, foi implementada uma função que carregue de um arquivo as informações necessárias para o preenchimento.

A função “carregarDeArquivo” recebe a lista e um arquivo. Se o arquivo for aberto com sucesso, a função declara um buffer linha para armazenar cada linha do arquivo e um ponteiro auxiliar regioAtual, inicializado como NULL, indicando em qual região será acrescentada as vinícolas correspondentes também presentes no arquivo. A leitura é feita através de um laço while, capturando uma linha por vez do arquivo, até que se atinja o fim dele (fgets retornando NULL). Cada linha lida tem seu caractere ‘\n’ removido através da combinação strcspn + atribuição de ‘\0’, garantindo que o conteúdo fique limpo para o processamento. Depois, a função lê o primeiro caractere e verifica se a linha em análise representa uma região “R” ou uma vinícola “V”, após tal identificação, lê o restante da linha a partir do comando sscanf com o formato "R; %[^\n]; %[^\n]" ou "V; %[^\n]; %[^\n]; %[^\n]; %d", capaz de separar e extrair os texto entre os “;”. Por fim, após a leitura, insere as informações da linha atual usando funções implementadas de inserção e passa para a próxima linha. Acabando o laço, o arquivo se fecha.

Note que o uso de sscanf se fez necessário, já que, diferente de scanf que lê dados digitados pelo usuário no teclado, ele lê e extrai dados de uma string já existente na memória – presente no arquivo.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "lista.h"

int main()
{
    ListaRegioes *lista = criarListaVazia();
    carregarDeArquivo(lista, "dados.txt");
}
```

```

void carregarDeArquivo(ListaRegioes *l, char *caminho)
{
    FILE *arq = fopen(caminho, "r");
    if (arq == NULL) {
        printf("Arquivo %s nao encontrado.\n", caminho);
        return;
    }

    char linha[MAX];
    Regioes *regiaoAtual = NULL;

    while (fgets(linha, MAX, arq) != NULL) {
        linha[strcspn(linha, "\n")] = '\0';

        if (linha[0] == 'R') {
            char nome[MAX], estado[MAX];
            sscanf(linha, "R;%[^;];%[^;]", nome, estado);
            inserirInicio(l, estado, nome);
            regiaoAtual = l->inicio;
        }
        else if (linha[0] == 'V' && regiaoAtual != NULL) {
            char nome[MAX], produtos[MAX], municipio[MAX];
            int ano;
            sscanf(linha, "V;%[^;];%[^;];%[^;];%d", nome, produtos, municipio, &ano);
            inserirVinInicio(regiaoAtual, nome, produtos, municipio, ano);
        }
    }

    fclose(arq);
}

```

Para permitir que o usuário interaja com as funções disponíveis no programa, é essencial a confecção de um menu indicando as ações que são possíveis de executar. O menu do nosso programa é arquitetado de forma hierárquica a partir de um sistema com menus secundários: com um menu principal que direciona o usuário para três submenus: Regiões, Vinícolas e Relatórios.

O menu principal controla, através de laço do-while que se repete até que o usuário escolha a opção 0 (Sair), a escolha dos menus secundários. As opções de ação são: acessar o submenu de regiões, acessar o submenu de vinícolas ou acessar o submenu de relatórios. Em todas as opções (exceto Sair), após a execução do submenu escolhido, a função exibe novamente o menu principal.

```

11
12     int opcao;
13     do {
14         printf("\n          MENU PRINCIPAL          \n");
15         printf("1 - Regioes\n");
16         printf("2 - Vinicolas\n");
17         printf("3 - Relatorios e buscas gerais\n");
18         printf("0 - Sair\n");
19         printf("Opcao: ");
20         scanf("%d", &opcao);
21

```

```

21
22         switch (opcao) {
23
24             case 1:
25                 menuRegioes(lista);
26                 break;
27
28             case 2:
29                 menuVinicolas(lista);
30                 break;
31
32             case 3:
33                 menuRelatorios(lista);
34                 break;
35
36             case 0:
37                 printf("Encerrando.\n");
38                 liberarLista(lista);
39                 break;
40
41             default:
42                 printf("Opcao invalida.\n");
43         }
44
45     } while (opcao != 0);
46
47     return 0;
48 }
49

```

Ainda falando sobre o menu principal, quando o usuário não quiser mais realizar as operações disponíveis e escolher a opção “Sair”, o programa se encerra. Por questões de boas práticas que evitem vazamento das memórias alocadas no código, é necessário liberar manualmente tudo que se foi alocado. Para isso, foi criada uma função do tipo void nomeada por “liberarLista” que é acionada quando o usuário escolhe encerrar o programa. A função recebe a lista de regiões. A partir da declaração de um ponteiro auxiliar, percorre-se todas as regiões alocadas na lista e, com a função “removerReg” já implementada, todas as regiões e suas respectivas vinícolas são liberadas. Por fim, também é liberado o descritor utilizado para guardar a lista de regiões.

```
529 void liberarLista(ListaRegioes *l)
530 {
531     if (l == NULL) {
532         return;
533     }
534     printf("Liberando lista... \n");
535
536     Regioes *aux = l->inicio;
537
538     while (aux != NULL) {
539         removerReg(l, aux->nomeReg);
540         aux = aux->proximo;
541     }
542
543     free(l);
544 }
545
```

Demonstrando os menus secundários.

O menu das regiões controla, através de um laço do-while também, todas as operações relacionadas à lista principal de regiões. As opções de ação são: inserir

região (chama inserirInicio), buscar região (chama buscaPorNomeReg), alterar dados de alguma região (chama alteraDadosReg), remover região (chama removerReg), listar regiões (chama listarRegioes), exibir número total de regiões (chama contaQuant) ou voltar (encerra o laço e retorna o controle ao menu principal).

```
546 void menuRegioes(ListaRegioes *l)
547 {
548
549
550     int opcao;
551     char nome[MAX], estado[MAX];
552
553
554     do {
555         printf ("\n      Regioes      \n");
556         printf ("1 - Inserir\n");
557         printf ("2 - Buscar\n");
558         printf ("3 - Alterar\n");
559         printf ("4 - Remover\n");
560         printf ("5 - Listar\n");
561         printf ("6 - Quantidade\n");
562         printf ("0 - Voltar\n");
563         printf ("Opcao: ");
564
565         scanf("%d", &opcao);
566
567         switch (opcao) {
568
569             case 1:
570
571                 printf("Nome: ");
572                 scanf(" %[^\\n]", nome);
573                 printf("Estado: ");
574                 scanf(" %[^\\n]", estado);
575                 inserirInicio(l, estado, nome);
576
577                 break;
```

```

578
579         case 2:
580
581             printf("Nome: ");
582             scanf(" %[^\\n]", nome);
583             buscaPorNomeReg(1, nome);
584
585             break;
586
587         case 3:
588
589             printf("Nome da regioao a alterar: ");
590             scanf(" %[^\\n]", nome);
591             alteraDadosReg(1, nome);
592
593             break;
594
595         case 4:
596
597             printf("Nome: ");
598             scanf(" %[^\\n]", nome);
599
600             removerReg(1, nome);
601
602             break;

```

```

604
605         case 5:
606
607             listarRegioes(1);
608
609             break;
610
611
612         case 6:
613
614             printf("Quantidade de regioes: %d\\n", contaQuantReg(1));
615
616             break;
617
618         case 0:
619
620             break;
621
622         default:
623             printf("Opcao invalida.\\n");
624     }
625
626     } while (opcao != 0);
627 }

```

O menu das vinícolas controla as operações relacionadas às listas secundárias. No entanto, ele exige que o usuário informe o nome de uma região que exista, onde serão realizadas as operações. As opções de ação são: inserir vinícola (chama `inserirVinInicio`), buscar vinícola (chama `buscaPorNomeVin`), alterar dados de alguma vinícola (chama `alteraDadosVin`), remover vinícola (chama `removerVin`), listar vinícolas (chama `listarVin`), exibir número total de vinícolas (chama `contaQuantVin`) ou voltar (encerra o laço e retorna o controle ao menu principal).

```
628
629 void menuVinicolas(ListaRegioes *l)
630 {
631     char nomeReg[MAX];
632     printf("\nNome da regioao: ");
633     scanf(" %[^\\n]", nomeReg);
634
635     Regioes *r = buscaPorNomeReg(l, nomeReg);
636
637     if (r == NULL) {
638         return;
639     }
640
641     int opcao;
642     char nome[MAX], produtos[MAX], municipio[MAX];
643     int ano;
644
645     do {
646         printf("\n      Vinicolas de %s      \\n", r->nomeReg);
647
648         printf("1 - Inserir\\n");
649         printf("2 - Buscar\\n");
650         printf("3 - Alterar\\n");
651         printf("4 - Remover\\n");
652         printf("5 - Listar\\n");
653         printf("6 - Quantidade\\n");
654         printf("0 - Voltar\\n");
655         printf("Opcao: ");
656
657         scanf("%d", &opcao);
658
```



```

659         switch (opcao) {
660             case 1:
661
662                 printf("Nome: ");
663                 scanf(" %[^\\n]", nome);
664
665                 printf("Produtos: ");
666                 scanf(" %[^\\n]", produtos);
667
668                 printf("Município: ");
669                 scanf(" %[^\\n]", municipio);
670
671                 printf("Ano de fundacao: ");
672                 scanf("%d", &ano);
673
674                 inserirVinInicio(r, nome, produtos, municipio, ano);
675
676                 break;
677
678             case 2:
679
680                 printf("Nome: ");
681                 scanf(" %[^\\n]", nome);
682                 buscaPorNomeVin(r, nome);
683
684                 break;
685

```

```

686             case 3:
687
688                 printf("Nome da vinicola a alterar: ");
689                 scanf(" %[^\\n]", nome);
690                 alteraDadosVin(r, nome);
691
692                 break;
693
694             case 4:
695
696                 printf("Nome: ");
697                 scanf(" %[^\\n]", nome);
698                 removerVin(r, nome);
699
700                 break;
701
702

```

```

702
703
704         case 5:
705
706             listarVin(r);
707
708             break;
709
710         case 6:
711
712             printf("Quantidade de vinícolas: %d\n", contaQuantVin(r));
713
714             break;
715
716         case 0:
717
718             break;
719
720         default:
721             printf("Opcao invalida.\n");
722     }
723
724     } while (opcao != 0);
725 }

```

Por fim, para realizar as operações definidas pelas funções adicionais, é preciso acessar o menu dos relatórios, também controlado por um laço do-while. As opções de ação são: buscar uma vinícola em todas as regiões (chama buscaGeral), contabilizar as vinícolas de todas as regiões (chama contaVinPorReg), exibir região com menos vinícolas (chama menorQuantVin), filtrar as vinícolas que pertencem ao mesmo município (chama filtrarVinPorMunicipio), contar quantas vinícolas pertencem ao mesmo município (chama contarVinPorMunicipio) ou voltar (encerra o laço e retorna ao menu principal). Note que as operações de filtro e de contagem por município só são executadas se a região informada for encontrada ($r \neq \text{NULL}$), evitando acesso a um ponteiro inválido.

```

727 void menuRelatorios(ListaRegioes *l)
728 {
729     int opcao;
730     char nome[MAX], nomeReg[MAX];
731
732     do {
733         printf("\n      Relatorios      \n");
734         printf("1 - Buscar vinicola em qualquer regioao\n");
735         printf("2 - Quantidade de vinicolas por regioao\n");
736         printf("3 - Regiao com menos vinicolas\n");
737         printf("4 - Filtrar vinicolas por municipio\n");
738         printf("5 - Contar vinicolas por municipio\n");
739         printf("0 - Voltar\n");
740         printf("Opcao: ");
741         scanf("%d", &opcao);
742
743         switch (opcao) {
744             case 1:
745                 printf("Nome da vinicola: ");
746                 scanf(" %[^\\n]", nome);
747                 buscaGeral(1, nome);
748                 break;
749
750             case 2:
751                 contaVinPorReg(1);
752                 break;
753
754             case 3:
755                 menorQuantVin(1);
756                 break;
757

```

```

757
758             case 4: {
759                 printf("Regiao: ");
760                 scanf(" %[^\\n]", nomeReg);
761                 Regioes *r = buscaPorNomeReg(1, nomeReg);
762                 if (r != NULL) {
763                     printf("Municipio: ");
764                     scanf(" %[^\\n]", nome);
765                     filtrarVinPorMunicipio(r, nome);
766                 }
767                 break;
768             }
769

```

```

769
770         case 5: {
771             printf("Regiao: ");
772             scanf(" %[^\\n]", nomeReg);
773             Regioes *r = buscaPorNomeReg(1, nomeReg);
774             if (r != NULL) {
775                 printf("Município: ");
776                 scanf(" %[^\\n]", nome);
777                 contarVinPorMunicípio(r, nome);
778             }
779             break;
780         }
781
782         case 0:
783             break;
784
785         default:
786             printf("Opcao invalida.\\n");
787     }
788
789     } while (opcao != 0);
790 }

```

EXEMPLOS DE USO

Considerando um arquivo “dados.txt” como entrada para exemplificar alguns usos do código escrito:

```
R;Vale dos Vinhedos;Rio Grande do Sul
V;Vinicola Aurora;Vinho Tinto, Vinho Branco;Bento Goncalves;1931
V;Casa Valduga;Espumante;Bento Goncalves;1875
R;Serra Gaucha;Rio Grande do Sul
V;Miolo;Vinho Tinto;Farroupilha;1897
```

Criaremos uma lista vazia e a preencheremos com as informações desse arquivo, basta criar uma variável do tipo ponteiro para estrutura do tipo ListaRegiões e chamar a função implementada para carregar dados de um arquivo:

```
ListaRegioes *lista = criarListaVazia();
carregarDeArquivo(lista, "dados.txt");
```

Agora, para começar os exemplos de uso de fato, realizaremos algumas das operações básicas na lista criada.

Em primeiro lugar, será realizada a operação de inserção de uma região na lista que, por conseguinte, cria uma região também. O usuário digitou o nome “Campanha Gaúcha” e o Estado “Rio Grande do Sul” para essa nova região, assim, chama-se, utilizando os parâmetros dados, a função `inserirInicio (lista, estado, nome)`. No entanto, essa função não imprime nenhuma saída, mas isso não nos impede de chamar a função `listarRegioes(lista)` para verificar se a inserção obteve sucesso.

Note que, como nosso código insere as regiões no começo da lista, a listagem das regiões cadastradas segue a ordem contrária de inserção, ou seja, o último elemento registrado é exposto primeiro e o primeiro, exposto por último.

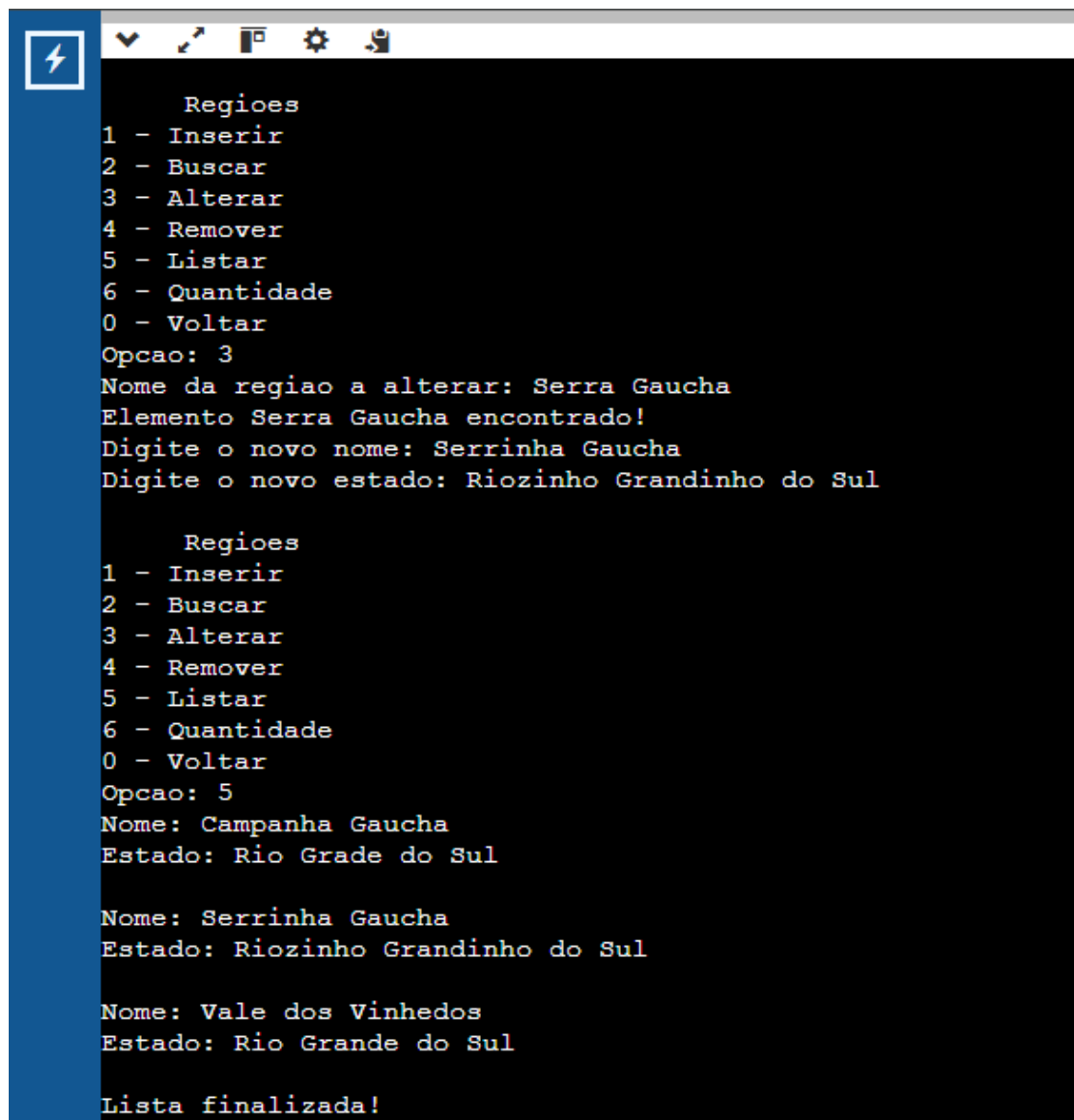
```
Regioes
1 - Inserir
2 - Buscar
3 - Alterar
4 - Remover
5 - Listar
6 - Quantidade
0 - Voltar
Opcao: 5
Nome: Campanha Gaucha
Estado: Rio Grade do Sul

Nome: Serra Gaucha
Estado: Rio Grande do Sul

Nome: Vale dos Vinhedos
Estado: Rio Grande do Sul

Lista finalizada!
```

Podemos também alterar dados de uma região já existente, por exemplo, alterar a região de nome “Serra Gaucha” para o nome “Serrinha Gaucha” e o Estado para “Riozinho Grandinho do Sul”. E, novamente para testar o êxito dessa operação, realizar a listagem das regiões cadastradas:



```
Regioes
1 - Inserir
2 - Buscar
3 - Alterar
4 - Remover
5 - Listar
6 - Quantidade
0 - Voltar
Opcao: 3
Nome da regioao a alterar: Serra Gaucha
Elemento Serra Gaucha encontrado!
Digite o novo nome: Serrinha Gaucha
Digite o novo estado: Riozinho Grandinho do Sul

Regioes
1 - Inserir
2 - Buscar
3 - Alterar
4 - Remover
5 - Listar
6 - Quantidade
0 - Voltar
Opcao: 5
Nome: Campanha Gaucha
Estado: Rio Grade do Sul

Nome: Serrinha Gaucha
Estado: Riozinho Grandinho do Sul

Nome: Vale dos Vinhedos
Estado: Rio Grande do Sul

Lista finalizada!
```

Aprofundando para a camada secundária: as listas de vinícolas, podemos realizar a operação de remover uma vinícola das adicionadas na região do Vale dos Vinhedos pelo arquivo “dados.txt”.

Observe a listagem de vinícolas da região do Vale dos Vinhedos antes da remoção e, depois, note a listagem de vinícolas dessa mesma região após a operação de remoção:

```

          Vinicolas de Vale dos Vinhedos
1 - Inserir
2 - Buscar
3 - Alterar
4 - Remover
5 - Listar
6 - Quantidade
0 - Voltar
Opcao: 5
Nome da vinicula: Casa Valduga
Produtos: Espumante
Municipio: Bento Goncalves
Ano de fundacao: 1875

Nome da vinicula: Vinicola Aurora
Produtos: Vinho Tinto, Vinho Branco
Municipio: Bento Gancalves
Ano de fundacao: 1931

Lista finalizada!

```

```

          Vinicolas de Vale dos Vinhedos
1 - Inserir
2 - Buscar
3 - Alterar
4 - Remover
5 - Listar
6 - Quantidade
0 - Voltar
Opcao: 4
Nome: Casa Valduga
Vinicola Casa Valduga encontrada!
Vinicola Casa Valduga encontrada!

          Vinicolas de Vale dos Vinhedos
1 - Inserir
2 - Buscar
3 - Alterar
4 - Remover
5 - Listar
6 - Quantidade
0 - Voltar
Opcao: 5
Nome da vinicula: Vinicola Aurora
Produtos: Vinho Tinto, Vinho Branco
Municipio: Bento Gancalves
Ano de fundacao: 1931

Lista finalizada!

```


Para finalizar a exemplificação dos usos do nosso código, realizaremos uma operação de consulta que necessite navegar tanto pela lista de regiões (principal) quanto pelas listas de vinícolas (secundárias).

Usando os mesmos dados iniciais carregados pelo arquivo, chamaremos a função que conta quantas vinícolas existem em cada região:

```
Relatorios
1 - Buscar vinicola em qualquer regioao
2 - Quantidade de vinicolas por regioao
3 - Regiao com menos vinicolas
4 - Filtrar vinicolas por municipio
5 - Contar vinicolas por municipio
0 - Voltar
Opcao: 2
Regiao Serra Gaucha : 1 Vinicolas
Regiao Vale dos Vinhedos : 2 Vinicolas
```

Também podemos chamar a função que conta quantas vinícolas de uma região, no caso do exemplo: Vale dos Vinhedos, pertencem ao mesmo município de Bento Gonçalves:

```
Relatorios
1 - Buscar vinicola em qualquer regioao
2 - Quantidade de vinicolas por regioao
3 - Regiao com menos vinicolas
4 - Filtrar vinicolas por municipio
5 - Contar vinicolas por municipio
0 - Voltar
Opcao: 5
Regiao: Vale dos Vinhedos
Elemento Vale dos Vinhedos encontrado!
Municipio: Bento Goncalves
Numero de Vinicolas no municipio: 2
```

CONCLUSÃO

Durante a confecção de um código, é comum encontrar algumas dificuldades que interrompem o processo de criação. No geral, as operações básicas de ambas as listas seguem o mesmo padrão de listas duplamente encadeadas, o que não foi trabalhoso para o grupo. No entanto, é fato de que desafios foram enfrentados em alguns pontos.

O primeiro empecilho do grupo foi pensar em como associar a lista secundária de vinícolas à lista principal de regiões e, após reflexões em conjunto, a solução se deu na atribuição de um papel duplo para estrutura do tipo “Regiões”. Além de exercer a função de encadeamento duplo da lista principal, usamos essa estrutura como nó descritor para o tipo “Vinícolas”, dessa forma, cada estrutura da lista de regiões está ligada uma lista de vinícolas. Inclusive, vale destacar que o grupo fortaleceu ainda mais essa associação ao criar uma lista vazia de vinícolas dentro da função “criarRegiao”.

Além disso, uma segunda dificuldade foi observada. No momento que se remove um elemento de uma lista duplamente encadeada, basta liberar a sua memória com o comando *free*. Porém, no caso desse trabalho, cada elemento da lista duplamente encadeada principal, caso não esteja vazia, carrega consigo outra lista duplamente encadeada. Com isso, era preciso liberar a lista associada junto com o elemento principal, o que, por uns instantes, passou batido pelo grupo e gerou um gasto desnecessário de memória no programa. Para resolver, a solução encontrada foi declarar mais um ponteiro auxiliar que removesse, dentro de um laço que se encerra quando os elementos da lista de vinícolas acaba, toda a lista de vinícolas.

Ainda, no final da confecção do programa, a criação de um bom menu gerou dúvida entre os membros do grupo. Devido ao número grande de operações que foram implementadas, um único menu contendo todas as ações dificultaria a interação do usuário com o código. Com isso, a solução encontrada foi separar o menu principal em menus menores, otimizando a manipulação dos comandos disponibilizados pelo programa.

Com toda certeza, é possível afirmar que o processo de codificação do trabalho se fez muito proveitoso para todos os membros do grupo. Foi o momento

perfeito para colocar em prática nossos estudos acerca da disciplina, bem como testar a absorção desse conhecimento. Com isso, entregar um código funcional de solução para problemática proposta prova que os objetivos do trabalho foram atingidos pelo grupo.